AI Model Development for Malware Detection

Assignment 1 Cyber Security and Artificial Intelligence

Submitted by	Alonzo A Kevin
Register number	3122255001021
Programme	BE Computer Science and Engineering
Year	2nd Year
Class	CSE-A
College	SSN College of Engineering

Dataset CIC MalMem 2022
Model Decision Tree Classifier
Platform Jupyter Notebook with Python

Abstract

This assignment develops a machine learning model to identify whether a Windows memory record is benign or malicious. I used the open CIC-MalMem-2022 benchmark dataset because it contains balanced examples of benign activity and obfuscated malware such as ransomware, spyware and Trojan horses. The work was completed in Python using pandas, scikit-learn and matplotlib. After cleaning the data, I selected the best 10 features using the ANOVA F-test, used an 80:20 stratified train-test split and trained a Decision Tree classifier. The model obtained 99.87% accuracy and a ROC-AUC of 0.9994 on the held-out test set. The result shows that memory-forensic features can separate the two classes very well in this benchmark.

Aim and objectives

The aim is to build and evaluate a supervised AI model for malware detection from memory-analysis features.

- Study an open benchmark cyber-security dataset.
- Clean and pre-process the raw CSV data.
- Select the top 10 useful features using a filter method.
- Train a Decision Tree model using an 80:20 split.
- Evaluate predictions using classification, error and ROC metrics.

Software and hardware requirements

Item	Used in this work
Programming language	Python 3
Platform	Jupyter Notebook
Libraries	pandas, NumPy, scikit-learn, matplotlib
Model	DecisionTreeClassifier
Dataset file	Obfuscated-MalMem2022.csv
Random seed	42

1 Dataset selection and understanding

1.1 Selected benchmark dataset

I selected CIC-MalMem-2022, released by the Canadian Institute for Cybersecurity at the University of New Brunswick. It is designed for detecting obfuscated malware through memory analysis. The original dataset contains 58,596 records, with 29,298 benign and 29,298 malicious records. The malicious class covers ransomware, spyware and Trojan horse families.

Official source: CIC-MalMem-2022 dataset page

1.2 Dataset structure

Property	Value
Raw records	58,596
Raw columns	57
Benign records	29,298
Malware records	29,298
Target column	Class
Example feature groups	Processes, DLLs, handles, modules, services and callbacks
Task type	Binary classification

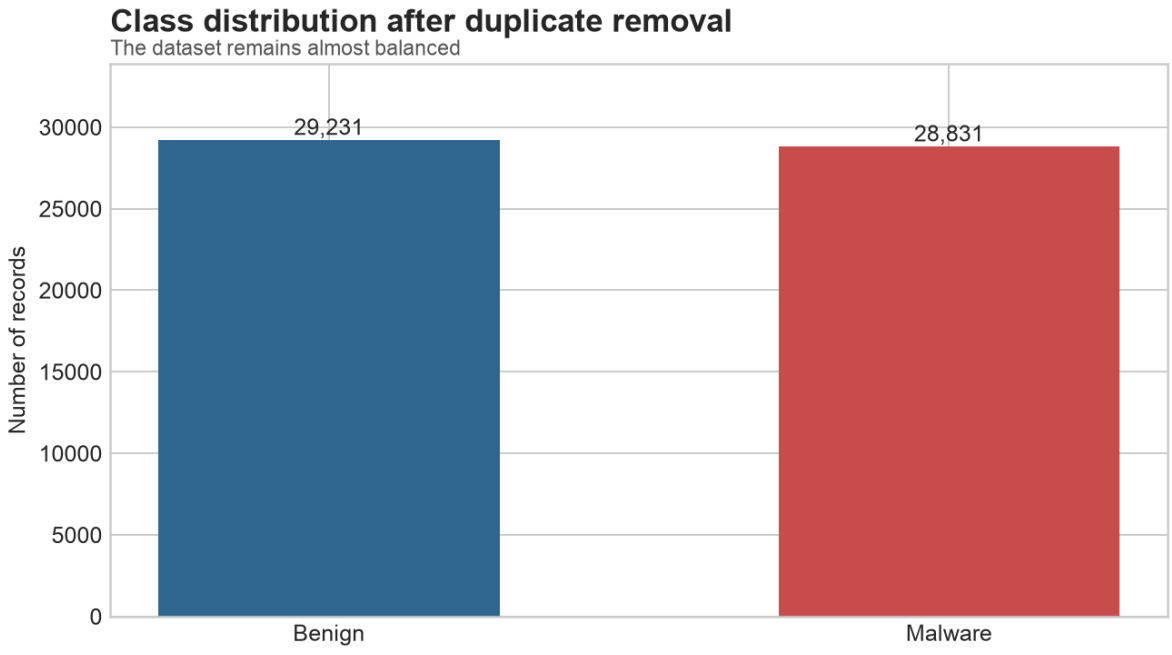


Figure 1 Class distribution after cleaning

1.3 Initial observations

- The raw classes are exactly balanced, so accuracy is meaningful but is still reported with precision, recall and F1 score.
- There are no null values in the supplied CSV, but the pipeline includes median imputation so that missing values can be handled safely.
- The Category column includes malware family information and was removed to avoid target leakage in binary classification.
- Three columns are constant and do not help the classifier.

2 Methodology and workflow

The workflow follows a normal supervised learning process. Data-dependent operations such as imputation, scaling and feature selection are fitted only on the training data through a scikit-learn Pipeline. This prevents information from the test set from influencing model training.



Figure 2 Methodology used for malware detection

1. Load the benchmark CSV file into a pandas DataFrame.
2. Inspect shape, data types, class counts, null values and duplicates.
3. Remove exact duplicates, constant columns and the Category leakage column.
4. Encode Benign as 0 and Malware as 1.
5. Split the cleaned records into 80% training and 20% testing using stratification.
6. Impute values, scale numerical features and select the top 10 features using ANOVA F-test.
7. Train a Decision Tree and calculate predictions, metrics and the ROC curve on the test set.

2.1 Why a Decision Tree was selected

A Decision Tree is suitable for this assignment because it is a supervised classification algorithm, handles non-linear rules, does not require a complicated mathematical model and can be visualised. I limited the tree depth to 8 and required at least 2 samples in each leaf to reduce unnecessary overfitting.

3 Data import and pre-processing

3.1 Importing and cleaning the data

The CSV file was imported with pandas. Column names were trimmed, infinite values were changed to missing values, and 534 exact duplicate rows were removed. The final working data contained 58,062 records.

Notebook cell Data loading and cleaning

```
df = pd.read_csv('data/Obfuscated-MalMem2022.csv')
df.columns = df.columns.str.strip()
df = df.replace([np.inf, -np.inf], np.nan).drop_duplicates()
y = df['Class'].map({'Benign': 0, 'Malware': 1})
X = df.drop(columns=['Class', 'Category'], errors='ignore')
constant_cols = [c for c in X if X[c].nunique(dropna=False) <= 1]
X = X.drop(columns=constant_cols)
```

```
Output
Raw shape: (58,596, 57)
Clean data: 58,062 rows and 52 predictor features
Missing values: 0    Exact duplicates remaining: 0
```

Screenshot 1 Python code for loading and cleaning the dataset

Check	Before	After
Rows	58,596	58,062
Raw or predictor columns	57 raw columns	52 predictors
Missing cells	0	0
Exact duplicate rows	534	0
Constant predictors	3	0
Class labels	Benign and Malware	0 and 1

3.2 Pre-processing decisions

Method	Implementation and reason
Null handling	Median imputation inside the pipeline. The current file has no nulls, but the step makes the model safer.
Duplicate handling	Exact duplicates were removed so repeated records do not influence both model learning and evaluation.
Label encoding	Benign = 0 and Malware = 1.
Leakage control	Category was removed because it describes the malware category and can reveal the target.
Constant removal	Three zero-variance columns were removed because they contain no separating information.

Method	Implementation and reason
Scaling	StandardScaler was fitted on training data. Scaling is not essential for a tree, but keeps the pipeline reusable.

4 Feature selection

I used SelectKBest with the ANOVA F-test, which is a filter-based univariate feature selection method. It checks each numeric feature separately and gives a high score when the feature values differ strongly between benign and malware classes. The selector was fitted only on the training portion. Exactly 10 features were retained as required.

Rank	Selected feature	ANOVA F score
1	dlllist.avg_dlls_per_proc	221,558.0
2	handles.nevent	153,963.7
3	handles.nthread	146,374.8
4	handles.nmutant	125,892.6
5	dlllist.ndlls	97,798.3
6	handles.nsection	87,721.8
7	pslist.avg_threads	77,621.0
8	ldrmodules.not_in_load	74,764.1
9	ldrmodules.not_in_mem	74,735.6
10	handles.ntimer	73,540.9

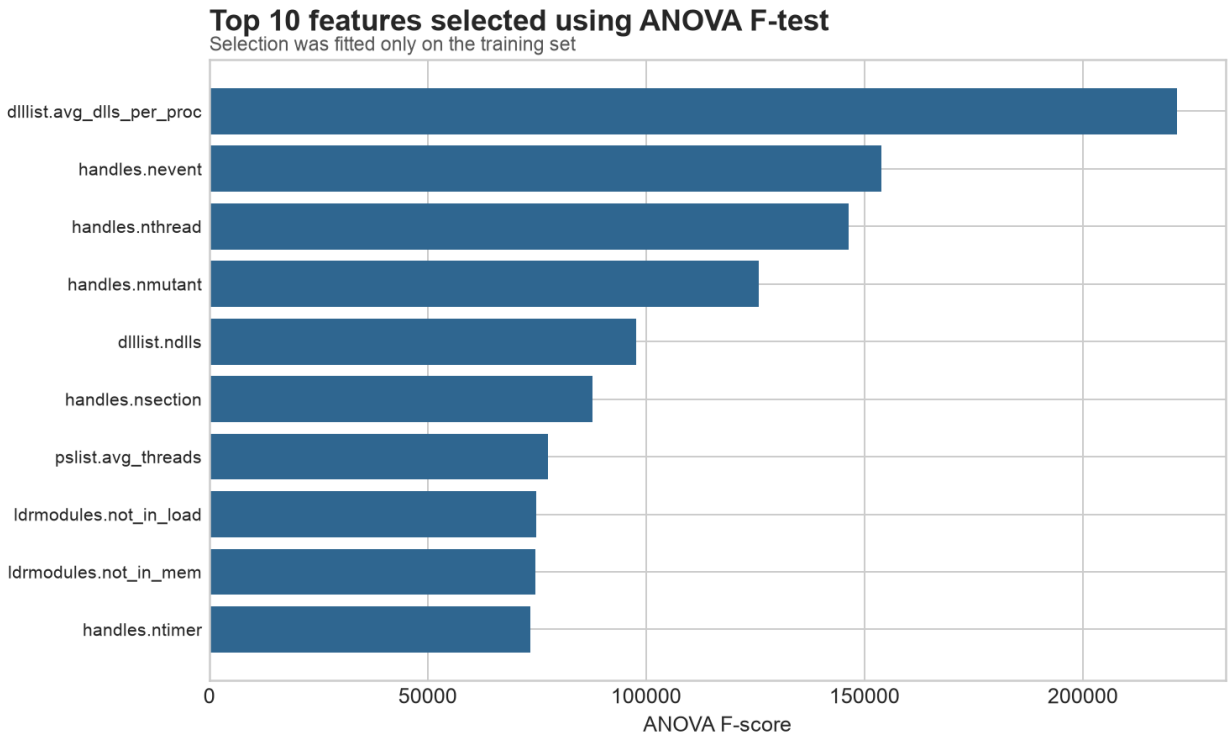


Figure 3 ANOVA scores of the selected features

4.1 Meaning of selected feature groups

Most selected predictors describe DLL loading and operating-system handles such as events, threads, mutants, sections and timers. These values represent how processes use memory and kernel objects. Malware can change these patterns because it injects code, creates hidden objects or behaves differently from normal software.

5 Train test split and model development

5.1 Train test split

The data was divided using an 80:20 stratified split. Stratification keeps almost the same benign-malware ratio in both parts. A fixed random state of 42 makes the result reproducible.

Partition	Records	Purpose
Training set	46,449	Fit imputer, scaler, feature selector and Decision Tree
Testing set	11,613	Final prediction and performance evaluation only

5.2 Model construction

Notebook cell Feature selection and Decision Tree

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y)

model = Pipeline([
    ('imputer', SimpleImputer(strategy='median')),
    ('scaler', StandardScaler()),
    ('selector', SelectKBest(score_func=f_classif, k=10)),
    ('classifier', DecisionTreeClassifier(
        max_depth=8, min_samples_leaf=2, random_state=42))
])
model.fit(X_train, y_train)
```

```
Output
Training rows: 46,449    Testing rows: 11,613
Selected features: 10
```

Screenshot 2 Pipeline, feature selection and Decision Tree training

Parameter	Value
criterion	Gini impurity
max_depth	8
min_samples_leaf	2
random_state	42
number of selected features	10

First three levels of the trained Decision Tree

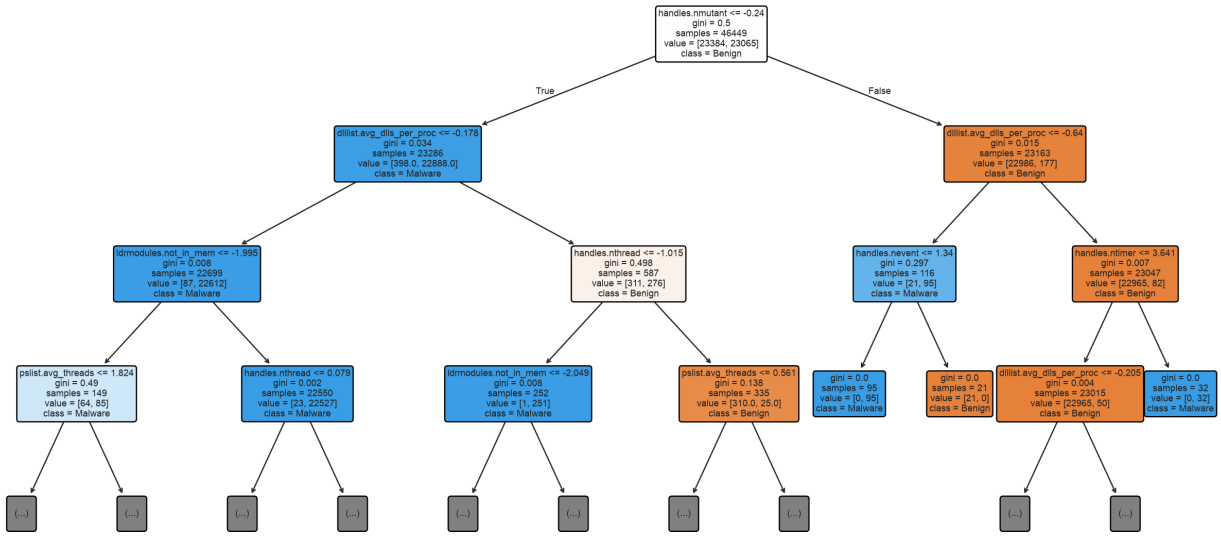


Figure 4 First three levels of the trained Decision Tree

6 Testing and sample predictions

The trained pipeline predicted the labels of the 11,613 unseen test records. The following table shows a small sample. The probability is the model's estimated probability that the record is malware.

No	Actual	Predicted	Malware probability
1	Benign	Benign	0.0000
2	Malware	Malware	1.0000
3	Benign	Benign	0.0000
4	Benign	Benign	0.0000
5	Malware	Malware	1.0000
6	Benign	Benign	0.0000
7	Malware	Malware	1.0000
8	Benign	Benign	0.0000
9	Malware	Malware	1.0000
10	Malware	Malware	1.0000

All ten examples above were classified correctly. The complete test set contained 15 errors: 5 benign records were marked as malware and 10 malware records were missed.

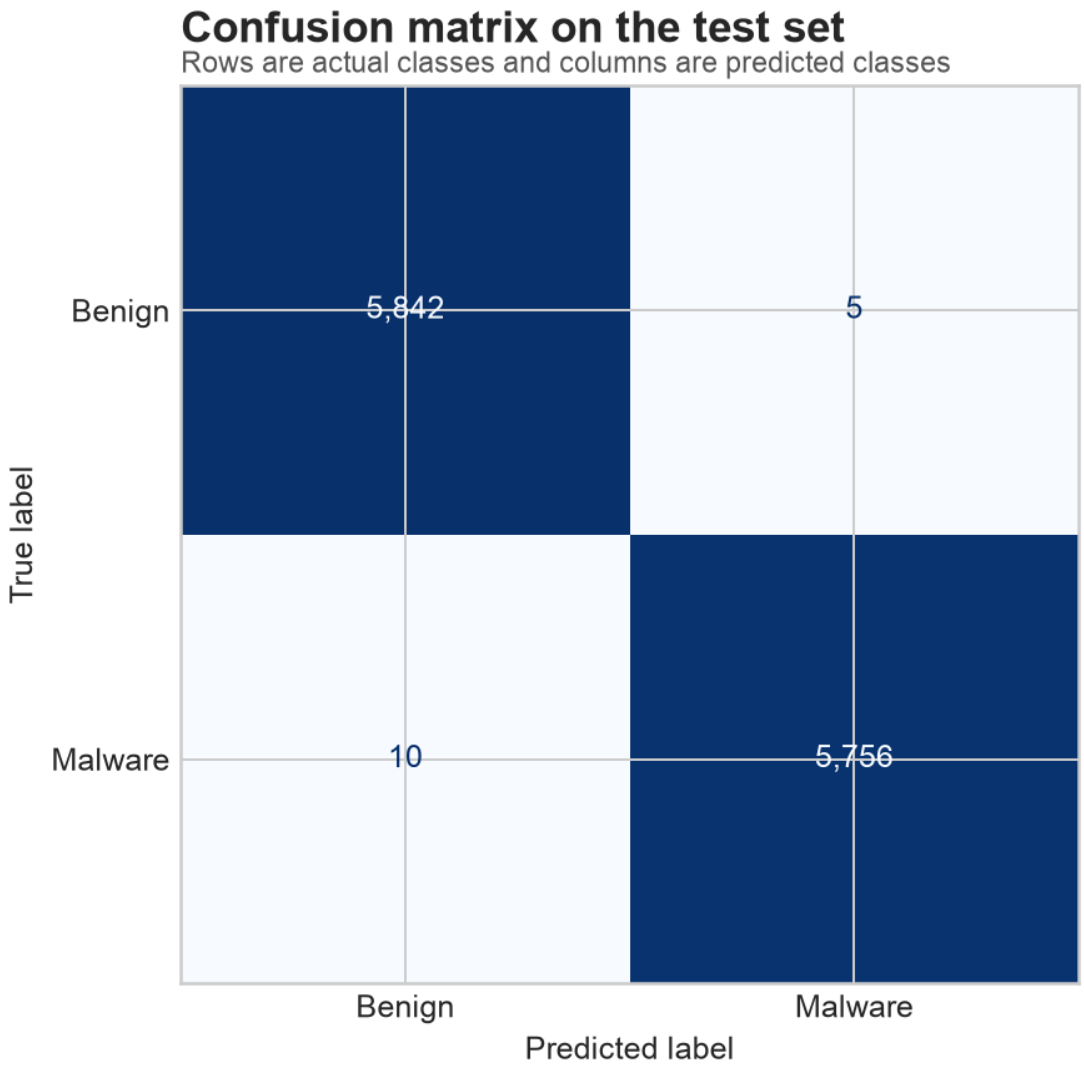


Figure 5 Confusion matrix for the complete test set

7 Performance evaluation

7.1 Classification metrics

Malware was treated as the positive class. Accuracy shows the overall correct proportion. Precision shows how many predicted malware records were actually malware. Recall shows how many true malware records were detected, and F1 score balances precision and recall.

Notebook cell Performance and error metrics

```
y_pred = model.predict(X_test)
y_prob = model.predict_proba(X_test)[:, 1]
print('Accuracy:', accuracy_score(y_test, y_pred))
print('Precision:', precision_score(y_test, y_pred))
print('Recall:', recall_score(y_test, y_pred))
print('F1:', f1_score(y_test, y_pred))
print('MAE:', mean_absolute_error(y_test, y_pred))
print('MSE:', mean_squared_error(y_test, y_pred))
print('ROC-AUC:', roc_auc_score(y_test, y_prob))
```

```
Output
Accuracy: 0.998708    Precision: 0.999132
Recall: 0.998266      F1: 0.998699
MAE: 0.001292         MSE: 0.001292
ROC-AUC: 0.999434
```

Screenshot 3 Python code and output for evaluation metrics

Metric	Value	Percentage
Accuracy	0.998708	99.87%
Precision	0.999132	99.91%
Recall	0.998266	99.83%
F1 score	0.998699	99.87%
ROC-AUC	0.999434	99.94%

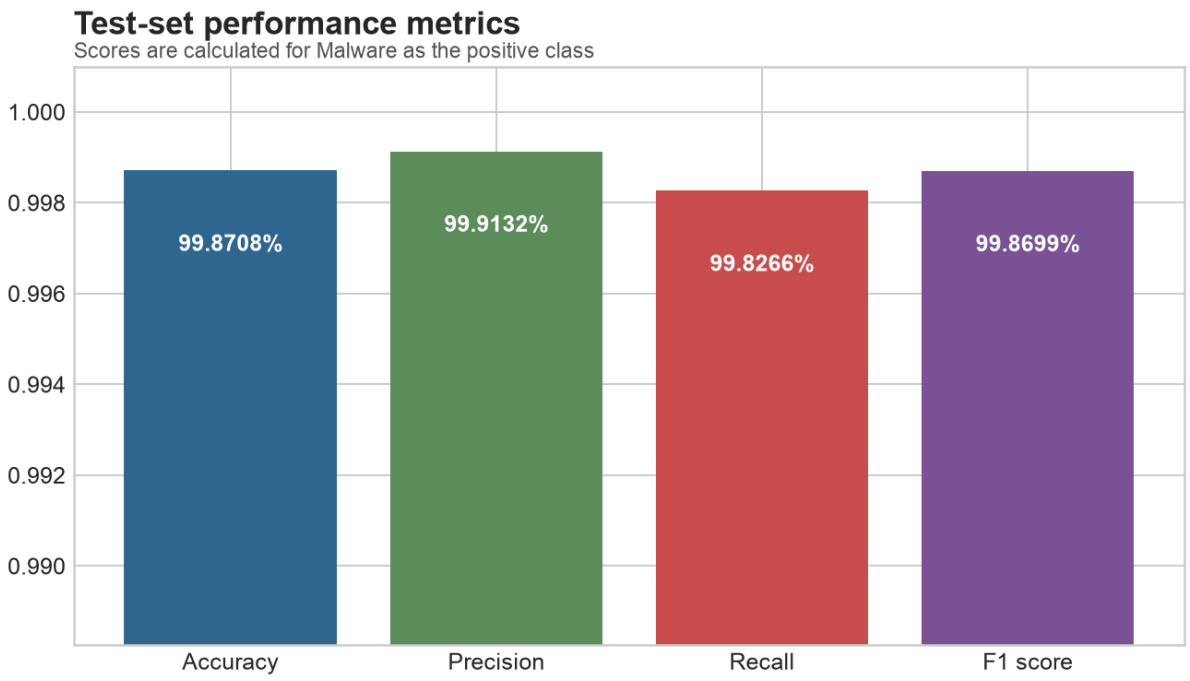


Figure 6 Main test performance metrics

7.2 Interpretation of confusion matrix

Result type	Count	Meaning
True negative	5,842	Benign records correctly predicted as benign
False positive	5	Benign records incorrectly flagged as malware
False negative	10	Malware records incorrectly predicted as benign
True positive	5,756	Malware records correctly detected

8 Error metrics

Mean Absolute Error and Mean Squared Error are mainly regression metrics, but the assignment asks for them. For binary labels 0 and 1 with hard class predictions, both become equal to the proportion of wrong predictions. Their values are low because only 15 out of 11,613 test records were misclassified.

Error metric	Value	Interpretation
Mean Absolute Error	0.001292	Average absolute difference between actual and predicted class label
Mean Squared Error	0.001292	Average squared difference between actual and predicted class label

For a classifier, precision, recall, F1 score, confusion matrix and ROC-AUC give more useful information than MAE or MSE.

9 ROC curve

The Receiver Operating Characteristic curve compares the true-positive rate with the false-positive rate at different probability thresholds. A curve near the top-left corner indicates strong separation. The area under the curve is 0.9994, which means the model ranks malware above benign samples very reliably on this test split.

Notebook cell ROC curve

```
fpr, tpr, thresholds = roc_curve(y_test, y_prob)
plt.plot(fpr, tpr, label=f'AUC = {roc_auc_score(y_test, y_prob):.4f}')
plt.plot([0, 1], [0, 1], '--', color='gray')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curve')
plt.legend()
plt.show()
```

Screenshot 4 Python code used to generate the ROC curve

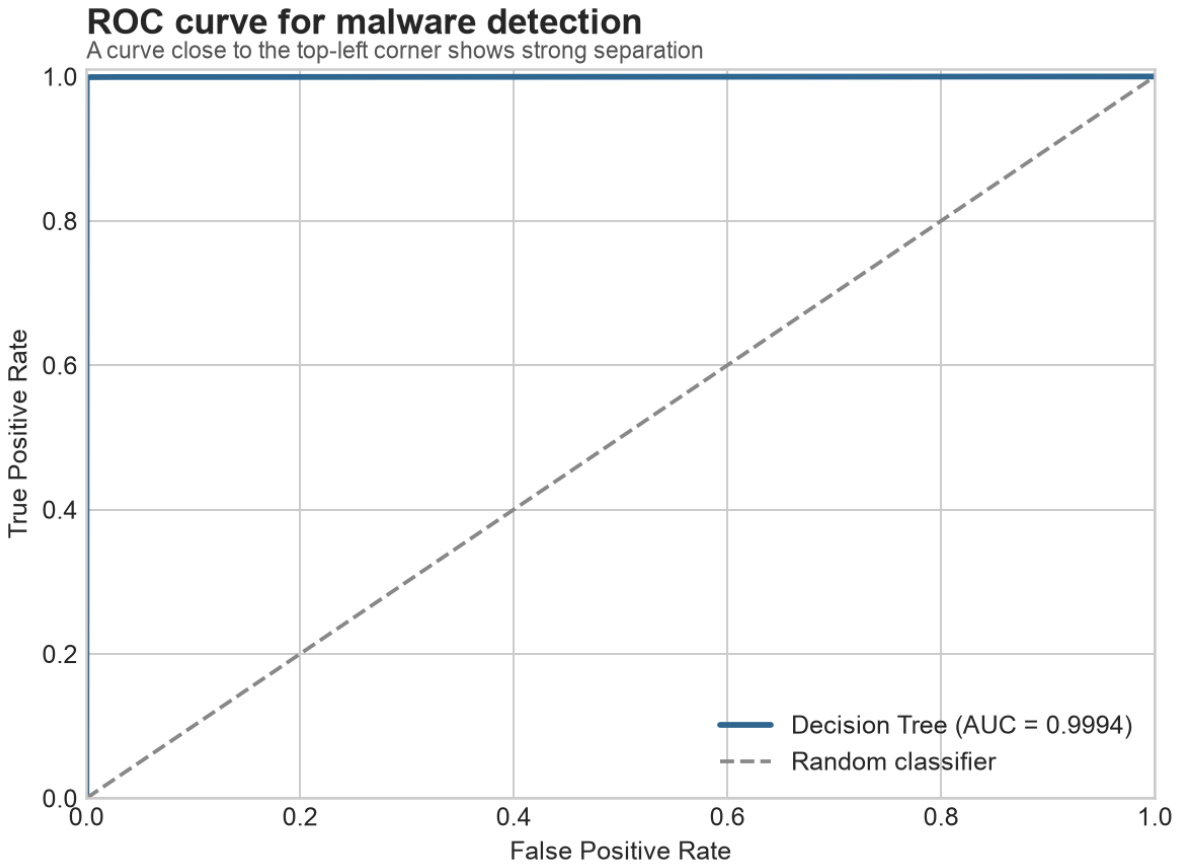


Figure 7 ROC curve of the final Decision Tree model

10 Final outcome and discussion

10.1 Outcome

The assignment successfully produced a binary malware detector using the CIC-MalMem-2022 dataset. The required stages were completed: data import, cleaning, encoding, scaling, filter-based top-10 feature selection, 80:20 splitting, supervised model training, prediction, classification metrics, error metrics and ROC analysis. The Decision Tree achieved 99.87% test accuracy and detected 5,756 of 5,766 malware records.

10.2 What I learned

- Pre-processing must be consistent for both training and future prediction.
- Feature selection reduces the number of inputs and keeps the model easier to understand.
- Recall is important in malware detection because a false negative leaves a malicious sample undetected.
- A confusion matrix explains model errors more clearly than accuracy alone.
- A pipeline helps prevent accidental test-data leakage during imputation, scaling and selection.

10.3 Limitations and future improvements

- The random row split may place related memory captures or malware families in both training and testing sets, so the score can be optimistic.
- The model was evaluated on one benchmark dataset. It should be checked on an independent and newer dataset before real deployment.
- A Decision Tree can overfit. Cross-validation and comparison with Random Forest, Extra Trees, SVM or gradient boosting can improve confidence.
- The current task is binary detection. Future work can classify ransomware, spyware and Trojan categories separately.
- A deployed system should monitor concept drift because malware behaviour changes over time.

Conclusion

This work shows a complete and reproducible machine learning workflow for malware detection. The selected memory features gave very strong separation between benign and malware records, and the Decision Tree produced high precision, recall and ROC-AUC. The model is suitable as an academic proof of concept, while external family-separated testing is still needed before it can be trusted as a real security product.

References

1. Canadian Institute for Cybersecurity, University of New Brunswick. Malware Memory Analysis CIC-MalMem-2022. https://www.unb.ca/cic/datasets/malmem-2022.html

2. T. Carrier, P. Victor, A. Tekeoglu and A. H. Lashkari. Detecting Obfuscated Malware using Memory Feature Engineering. ICISSP 2022. https://www.unb.ca/cic/datasets/malmem-2022.html

3. scikit-learn documentation. DecisionTreeClassifier. https://scikit-learn.org/stable/modules/generated/sklearn.tree.DecisionTreeClassifier.html

4. scikit-learn documentation. SelectKBest and f_classif. https://scikit-learn.org/stable/modules/feature_selection.html

5. scikit-learn documentation. Classification metrics and ROC curve. https://scikit-learn.org/stable/modules/model_evaluation.html

Appendix A Reproducibility

The accompanying Jupyter Notebook contains the complete Python workflow and saved outputs. To reproduce the analysis, place the notebook in the assignment_report folder, keep the dataset in the project data folder and run all cells from top to bottom.

File	Purpose
Assignment_1_Malware_Detection.ipynb	Full notebook with code, outputs and ROC figure
top_10_features.csv	Selected feature names and ANOVA scores
sample_predictions.csv	Example test predictions
results.json	Machine-readable experiment summary